ruby/migration_trips_check_constraint.rb
```
class AddCheckConstraintTripsCompletedAt < ActiveRecord::Migration[7.0]
  def change
    add_check_constraint :trips, "completed_at > created_at",
      name: "trips_completed_after_created"
  end
end
```

In the same way as the trips table, the Vehicle Reservation start and end times should make sense chronologically where the ends_at time should always be *after* the starts_at time.

This Check constraint for Vehicle Reservations has not been added to Rideshare. See if you can add it to your copy! You could even fork the Rideshare repo and continue to evolve your own database schema.

If there are rows that violate the constraint, PostgreSQL will display an error message:

sql/error_constraint_violation.sql
```
ERROR: check constraint "trips_completed_after_created"
of relation "trips" is violated by some row
```

You've now seen how Check constraints can help make sure your timestamps are in the correct order. This might help you avoid a data consistency bug where the front-end client application posts invalid data, by making sure the data is rejected.

These are relatively straightforward constraints, but what about one that's a bit more complicated?

Imagine that Vehicle Reservations must be a minimum of 30 minutes in length.

Since having a reservation that's too short is a significant business problem for Rideshare, you decide to create a Check constraint to help enforce minimum reservation lengths.

One strategy with a SQL expression in a Check constraint definition is below. This builds on what you did earlier, making sure the ends_at column is at least 30 minutes after the starts_at column value. Both of these columns prevent NULLs.

Put this SQL statement into psql for the Rideshare database:

sql/check_constraint_reservation_length.sql
```
ALTER TABLE vehicle_reservations
ADD CONSTRAINT vehicle_reservation_minimum_length
CHECK (ends_at >= (starts_at + INTERVAL '30 minutes'));
```